

面向对象编程基础技能之

对象比较

北京理工大学计算机学院
金旭亮





长幼有序



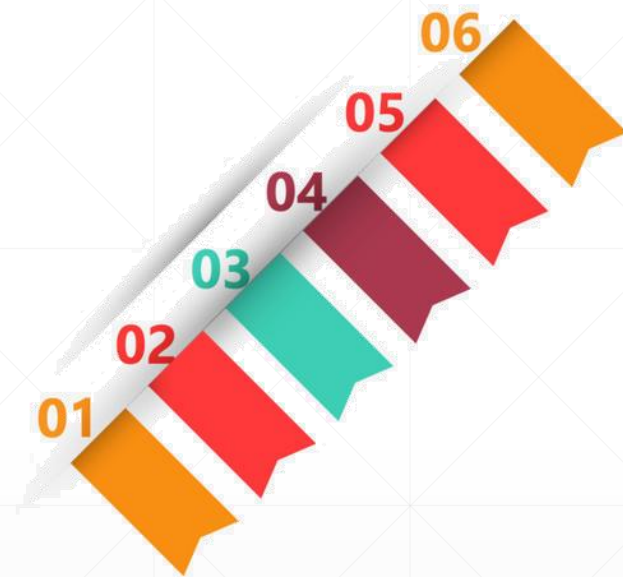
大小相依

“引用比较”与“内容比较”

- 对于基本数据类型的变量，可以使用“==”，“>”和“<”进行判等和大小比较，但除了“==”之外，“>”和“<”是不能直接用于引用类型的变量的。
- 经过前面的学习，我们已经知道，施加于两个对象变量之上的“==”，实际上是判断这两个对象变量是否引用同一个对象。
- 在实际开发中，我们经常需要比对某两个对象的“内容”，比如你已经有了一个对象，想在另一个对象集合中找到是否有“内容一样”的对象。
- 这里所说的对象的“内容”，主要指对象的字段值。
- 由于字段值在不同的时刻可能会改变，在某一特定时刻，对象所有字段值的集合，称为对象在这一时刻的“状态”。

我们将要学习什么？

在本讲中，我们将学习如何比较两个对象的“大小”，以便能在此基础上编写代码，实现对一个对象集合进行查找和排序。



对象的比较：Comparable接口

JDK中为了能比较两个对象的大小，定义了以下接口：

```
public interface Comparable {  
    int compareTo(Object other);  
}
```

compareTo() 方法的返回值，约定如下：

1. 对象X和Y相等，返回 “0”
2. 对象X<Y，返回 “-1”
3. 对象X>Y，返回 “1”

泛型化的Comparable接口

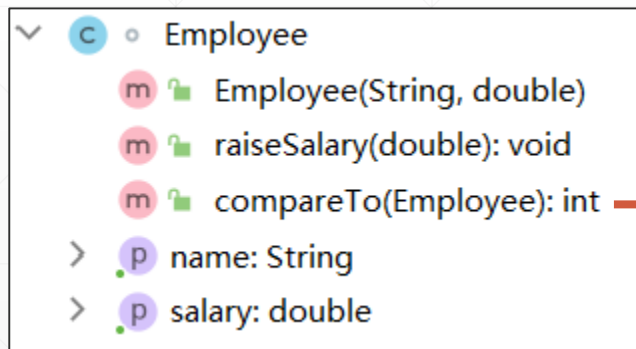
引入泛型特性后，JDK中又增加了一个泛型化的接口，老的版本现在已经不再推荐使用：

```
public interface Comparable<T> {  
    int compareTo(T other);  
}
```

动手动脑



- 请阅读示例：EmployeeSortTest.java
- 此示例按照工资高低确定两个员工对象“谁大谁小”，并以此为依据排序输出放在一个数组中的三个Employee对象。
- 请通过阅读示例源码理解对象比较大小的基本原理



```
public int compareTo(Employee other)
{
    if (salary < other.salary) return -1;
    if (salary > other.salary) return 1;
    return 0;
}
```

JDK中的实例

- JDK中凡是支持大小比较的类型（比如Integer），都实现了Comparable<T> 接口。

```
public final class Integer extends Number implements Comparable<Integer> {  
    .....  
    private final int value;  
  
    public int compareTo(Integer anotherInteger) {  
        return compare(this.value, anotherInteger.value);  
    }  
  
    public static int compare(int x, int y) {  
        return (x < y) ? -1 : ((x == y) ? 0 : 1);  
    }  
    .....  
}
```

摘自JDK中Integer类的源码

区分“==”与“equals”

- “==” 施加于引用类型变量，是比较两个对象变量是否引用同一对象。如果需要比对对象的“内容（即各字段的值）”，通常是调用对象的equals方法。
- equals方法由Object类所定义，其默认实现如下：

```
public boolean equals(Object obj) {  
    // 默认情况下是比较对象引用，而不是对象“内容”  
    return (this == obj);  
}
```

重写equals()方法

子类可根据实际情况，“**重写 (Override)**” Object 类的 equals 方法。

```
class MyClass {  
  
    public int InnerValue;  
    public String InnerString;  
  
    public boolean equals(Object obj) {  
        boolean result = obj instanceof MyClass;  
        if (!result) {  
            return false;  
        } else {  
            MyClass other = (MyClass) obj;  
            return (other.InnerValue == this.InnerValue)  
                && (other.InnerString.equals(this.InnerString));  
        }  
    }  
}
```

重写equals方法，其实就是要你确定一下“两个对象怎样才算相等”。

IsTwoObjectEquals.java

重写equals()的必要性

- JDK的许多集合类型，比如ArrayList，在查找元素时，会调用元素的equals()方法以确定当前元素是不是要找的那个。

```
public int indexOf(Object o) {  
    if (o == null) {  
        for (int i = 0; i < size; i++)  
            if (elementData[i]==null)  
                return i;  
    } else {  
        for (int i = 0; i < size; i++)  
            if (o.equals(elementData[i]))  
                return i;  
    }  
    return -1;  
}
```

摘自JDK中ArrayList.indexOf()
方法的源码

如果你写了一个类，把对象放到ArrayList集合中，那么，你需要重写equals方法，才能在集合中使用indexOf()方法查找到特定的对象，这个就是为什么你在定义类时，需要重写equals()方法的原因。

重写equals方法的要求

- **自反性 (reflexive)** : `x.equals(x) = true`
- **对称性 (symmetric)** : 如果 `x.equals(y) = true`, 那么 `y.equals(x) = true`
- **传递性 (transitive)** : 如果 `x.equals(y) = true` 并且 `y.equals(z) = true`, 那么 `x.equals(z) = true`.
- **一致性 (consistent)** : 只要对象的状态没有发生改变, `x.equals(y)` 的多次调用应该返回一致的结果
- **`x.equals(null) = false`**
- **equals()方法要与compareTo()方法的返回值一致。** 当equals()方法返回true时, compareTo()方法应该返回0。所以, 为了保持两个方法结果的一致性, 可以在重写equals()方法时, 直接调用compareTo()方法, 以避免相同的对象比较规则重复写两遍。

一个容易忽略的问题



- 要重写Object的equals方法，注意其参数类型必须是“Object”，如果是其它类型，就是“重载（overload）”，重载equals()方法，并不能保证这个对象放到JDK的各种集合之后，就能支持“查找”这样的建立于“判等”基础之上的“二级”功能。

```
public boolean equals(Object obj) {  
    //...  
}
```

- 另外，为了让对象能放入JDK所提供的各种集合中，通常还需要重写hashCode()方法，因为这些集合在内部可能会需要依据对象的hashCode值进行定位。

一个可以“判等”的类的示例

```
public class MyClass{  
    private long myID;  
  
    public MyClass() {  
        myID = System.nanoTime();  
    }  
  
    public boolean equals(Object o) {  
        if(o instanceof MyClass){  
            MyClass other = (MyClass)o;  
            return myID == other.myID;  
        }  
        else { return false;}  
    }  
  
    public int hashCode() {  
        return (int)myID;  
    }  
}
```

使用当前时间（纳秒数）作为对象标识

重写equals方法，使本对象支持查找功能，这里我们约定，只要Id值一样，这两个对象就“相等”。

重写hashCode方法，使本对象支持哈希定位

两个不同的对象（不管它们是否“内容相等”），应该让它拥有不同的hashCode。hashCode的生成，有各种典型的算法，详情请百度。



如果类是你自己写的，那么，你当然可以给它重写 `equals()` 方法和 `hashCode()` 方法，以便让它支持大小比较和判等功能。

但如果这个类是其他人写的，你无法修改它的源代码，又该怎么办呢？

解决之道

可以编写一个实现Comparator接口的类，实现compare和equals方法，这个类的基本职责就是“判断两个对象谁大谁小”，类似于体育比赛中的“裁判员”.....

```
public interface Comparator<T> {  
    int compare(T o1, T o2);  
    boolean equals(Object obj);  
    //...  
}
```



自定义“外部”对象比较规则示例

示例PersonTest.java中使用
AgeComparator依据对象的age字段
值比较两个Person对象的大小

```
//按照年龄比较两个Person对象的大小
class AgeComparator implements Comparator<Person> {
    @Override
    public int compare(Person o1, Person o2) {
        int age1 = o1.getAge();
        int age2 = o2.getAge();
        if (age1 > age2) {
            return 1;
        } else if (age1 < age2) {
            return -1;
        }
        return 0;
    }
}
```

排序:

```
Arrays.sort(persons, new AgeComparator());
```

动手动脑



- 编写一个泛型方法，能对一个对象数组进行排序（要求对象实现 `Comparable<T>`）接口。
- 设计一个学生类（包括学生姓名、年龄两个字段），创建若干个学生对象放到数组或 `ArrayList` 中。
 1. 编写方法在此数组(或 `ArrayList`)中按照学生姓名查找相应的对象。
 2. 支持按年龄进行升序和降序输出。